Web Services - Assignment 1 (B00157067)

GitHub Repo Link - https://github.com/IsraelAdeniran/WebServices-Assignment

1. Fast API Python Code

```python
# Web Services Assignment - FastAPI Application
# Student: B00157067

import os
import requests
from fastapi import FastAPI, HTTPException
from pymongo import MongoClient
from pydantic import BaseModel
from dotenv import load_dotenv
from prometheus_fastapi_instrumentator import Instrumentator

# Load environment variables and connect to MongoDB
load_dotenv(dotenv_path="../.env")

app = FastAPI()
Instrumentator().instrument(app).expose(app)

client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["products"]


# Pydantic model for product validation
class Product(BaseModel):
    ProductID: int
    Name: str
    UnitPrice: float
    StockQuantity: int
    Description: str


# Convert MongoDB ObjectId to string
def fix_id(doc):
    doc["_id"] = str(doc["_id"])
    return doc


# Get a single product by ProductID
@app.get("/getSingleProduct/{product_id}")
def get_single_product(product_id: int):
    product = collection.find_one({"ProductID": product_id})
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")
    return fix_id(product)


# Get all products
@app.get("/getAll")
def get_all():
    products = list(collection.find())
    return [fix_id(p) for p in products]
```

```python
# Add a new product
@app.post("/addNew")
def add_new(product: Product):
    existing = collection.find_one({"ProductID": product.ProductID})
    if existing:
        raise HTTPException(status_code=400, detail="ProductID already
exists")
    collection.insert_one(product.dict())
    return {"message": "Product added successfully"}


# Delete a product by ProductID
@app.delete("/deleteOne/{product_id}")
def delete_one(product_id: int):
    result = collection.delete_one({"ProductID": product_id})
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Product not found")
    return {"message": "Product deleted successfully"}


# Get products where name starts with a given letter
@app.get("/startsWith/{letter}")
def starts_with(letter: str):
    if len(letter) != 1:
        raise HTTPException(status_code=400, detail="Please provide a
single letter")
    products = list(collection.find({"Name": {"$regex": f"^{letter}",
"$options": "i"}}))
    return [fix_id(p) for p in products]


# Get products from start ID to end ID in batches of 10
@app.get("/paginate")
def paginate(start_id: int, end_id: int):
    products = list(collection.find(
        {"ProductID": {"$gte": start_id, "$lte": end_id}}
    ).limit(10))
    if not products:
        raise HTTPException(status_code=404, detail="No products found in
that range")
    return {
        "start_id": start_id,
        "end_id": end_id,
        "results": [fix_id(p) for p in products]
    }


# Convert product price from USD to EUR using live exchange rate
@app.get("/convert/{product_id}")
def convert(product_id: int):
    product = collection.find_one({"ProductID": product_id})
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")
    response = requests.get("https://api.exchangerate-
api.com/v4/latest/USD")
    rate = response.json()["rates"]["EUR"]
    eur_price = round(product["UnitPrice"] * rate, 2)
    return {
        "Name": product["Name"],
        "UnitPrice_USD": product["UnitPrice"],
```

```
        "UnitPrice_EUR": eur_price,
        "ExchangeRate": rate
    }
```

2. Unit Test Code

main.py collection.json

```
{
  "info": {
    "_postman_id": "131eec99-869a-4b26-9e6f-13379cef8343",
    "name": "Web Services Assignment Tests",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json",
    "_exporter_id": "38672995"
  },
  "item": [
    {
      "name": "Get Single Product",
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test(\"Status code is 200\", function () {\r",
              "    pm.response.to.have.status(200);\r",
              "});"
            ],
            "type": "text/javascript",
            "packages": {},
            "requests": {}
          }
        }
      ],
      "request": {
        "method": "GET",
        "header": [],
        "url": {
          "raw": "http://host.docker.internal:8000/getSingleProduct/1001",
          "protocol": "http",
          "host": [
            "host.docker.internal"
          ],
          "port": "8000",
          "path": [
            "getSingleProduct",
            "1001"
          ]
        }
      },
      "response": []
    },
    {
      "name": "Get All Products",
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test(\"Status code is 200\", function () {\r",
              "    pm.response.to.have.status(200);\r",
              "});"
            ],
            "type": "text/javascript",
            "packages": {}
          }
        }
```

main.py {} collection.json


```
      "item": [
        {
          "event": [
            }
          ],
          "request": {
            "method": "GET",
            "header": [],
            "url": {
              "raw": "http://host.docker.internal:8000/getAll",
              "protocol": "http",
              "host": [
                "host.docker.internal"
              ],
              "port": "8000",
              "path": [
                "getAll"
              ]
            }
          },
          "response": []
        },
        {
          "name": "Add New Product",
          "event": [
            {
              "listen": "test",
              "script": {
                "exec": [
                  "pm.test(\"Status code is 200\", function () {\r",
                  "    pm.response.to.have.status(200);\r",
                  "});"
                ],
                "type": "text/javascript",
                "packages": {}
              }
            }
          ],
          "request": {
            "method": "POST",
            "header": [],
            "body": {
              "mode": "raw",
              "raw": "{\r\n    \"ProductID\": 9999,\r\n    \"Name\": \"Test Product\",\r\n    \"UnitPrice\": 9.99,\r\n    \"StockQuantity\": 5,\r\n    \"Description\": \"A test product\"\r\n}",
              "options": {
                "raw": {
                  "language": "json"
                }
              }
            },
            "url": {
              "raw": "http://host.docker.internal:8000/addNew",
              "protocol": "http",
              "host": [
                "host.docker.internal"
              ],
              "port": "8000",
              "path": [
```

3. Docker Configuration

Jenkinsfile

```
// Web Services Assignment - Jenkins Pipeline
// Student: B00157067

pipeline {
    agent any

    environment {
        MONGO_URI = credentials('MONGO_URI')
    }

    stages {

        // Pull the latest code from GitHub
        stage('Clone') {
            steps {
                git branch: 'main', url:
'https://github.com/IsraelAdeniran/WebServices-Assignment'
            }
        }

        // Build the Docker image
        stage('Build') {
            steps {
                sh 'docker build -t webservices-app .'
            }
        }

        // Run the Docker container with credentials passed securely
        stage('Run') {
            steps {
                sh 'docker rm -f webservices-container || true'
                sh 'docker run -d -p 8000:8000 --name webservices-container
-e MONGO_URI=$MONGO_URI -e DB_NAME=productsDB webservices-app'
                sh 'sleep 10'
            }
        }

        // Run Newman tests
        stage('Test') {
            steps {
                sh 'newman run tests/collection.json'
            }
        }

        // Stop and remove the container
        stage('Stop') {
            steps {
                sh 'docker stop webservices-container'
                sh 'docker rm webservices-container'
            }
        }

        // Generate README.txt and zip everything
        stage('Package') {
            steps {
                sh '''echo "Web Services Assignment API
```

```
Endpoints:
- GET  /getSingleProduct/{product_id} - Returns a single product by ID
- GET  /getAll - Returns all products
- POST /addNew - Adds a new product (ProductID, Name, UnitPrice,
StockQuantity, Description)
- DELETE /deleteOne/{product_id} - Deletes a product by ID
- GET  /startsWith/{letter} - Returns products starting with a letter
- GET  /paginate?start_id=1001&end_id=1010 - Returns up to 10 products
between IDs
- GET  /convert/{product_id} - Returns product price converted to EUR
- GET  /metrics - Prometheus monitoring metrics
- GET  /docs - FastAPI Swagger documentation
Build date: $(date)" > README.txt'''
                sh 'zip -r complete-$(date +"%Y-%m-%d_%H-%M-%S").zip . -x
"*.git*" -x ".venv/*" -x "venv/*"'
            }
        }
    }

    post {
        always {
            archiveArtifacts artifacts: '*.zip', fingerprint: true
        }
    }
}
```

Dockerfile

```
# Use Ubuntu as the base image
FROM ubuntu:22.04

# Author
LABEL authors="adefola"

# Install Python and pip
RUN apt-get update && apt-get install -y python3 python3-pip

# Set the working directory
WORKDIR /app

# Copy requirements and install dependencies
COPY app/requirements.txt .
RUN pip3 install -r requirements.txt

# Copy the app folder
COPY app/ .

# Expose the port
EXPOSE 8000

# Run the application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4. API Endpoints Screenshots

Get Single Product

GET /getSingleProduct/{product_id} Get Single Product

Parameters

Cancel

Name	Description
product_id * required integer (path)	1001

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/getSingleProduct/1001' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/getSingleProduct/1001
```

Server response

Code	Details
200	Response body

```
{
  "_id": "69cadf9eda6b3bf6af3bfe29",
  "ProductID": 1001,
  "Name": "NVIDIA RTX 4090",
  "UnitPrice": 1599.99,
  "StockQuantity": 12,
  "Description": "High-end GPU with 24GB VRAM for 4K gaming."
}
```

Download

Response headers

```
content-length: 174
content-type: application/json
date: Thu,02 Apr 2026 17:47:56 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

Get All Products

GET /getAll Get All

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/getAll' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/getAll
```

Server response

Code	Details
200	Response body


```
[
  {
    "_id": "69cadf9eda6b3bf6af3bfe29",
    "ProductID": 1001,
    "Name": "NVIDIA RTX 4090",
    "UnitPrice": 1599.99,
    "StockQuantity": 12,
    "Description": "High-end GPU with 24GB VRAM for 4K gaming."
  },
  {
    "_id": "69cadf9eda6b3bf6af3bfe2a",
    "ProductID": 1002,
    "Name": "AMD Ryzen 9 7950X",
    "UnitPrice": 549,
    "StockQuantity": 25,
    "Description": "16-core 32-thread enthusiast desktop processor."
  },
  {
    "_id": "69cadf9eda6b3bf6af3bfe2b",
    "ProductID": 1003,
    "Name": "Samsung 990 Pro 2TB",
    "UnitPrice": 179.99,
    "StockQuantity": 40,
    "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads."
  },
  {
    "_id": "69cadf9eda6b3bf6af3bfe2c",
    "ProductID": 1004,
```


Download

Response headers

```
content-length: 34747
content-type: application/json
date: Thu,02 Apr 2026 17:49:41 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Add New Products

POST /addNew Add New

Parameters

Cancel Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "ProductID": 9998,
  "Name": "Test Product",
  "UnitPrice": 9.99,
  "StockQuantity": 5,
  "Description": "A test product"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/addNew' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "ProductID": 9998,
  "Name": "Test Product",
  "UnitPrice": 9.99,
  "StockQuantity": 5,
  "Description": "A test product"
}'
```

Request URL

```
http://localhost:8000/addNew
```

Server response

Code	Details
200	Response body

```
{
  "message": "Product added successfully"
}
```

Download

Response headers

```
content-length: 40
content-type: application/json
date: Thu,02 Apr 2026 17:52:05 GMT
server: uvicorn
```

Responses

Code	Description	Links

Delete One Product

DELETE /deleteOne/{product_id} Delete One

Parameters

Cancel

Name	Description
product_id * required integer (path)	9998

Execute | Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:8000/deleteOne/9998' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/deleteOne/9998
```

Server response

Code	Details

200	Response body
	{
	"message": "Product deleted successfully"
	}
	Download
	Response headers
	content-length: 42
	content-type: application/json
	date: Thu,02 Apr 2026 17:53:42 GMT
	server: uvicorn

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value | Schema "string"	No links

Starts With

GET /startsWith/{letter} Starts With

Parameters

Cancel

Name	Description
letter * required string (path)	N

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/startsWith/N' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/startsWith/N
```

Server response

Code	Details
200	Response body


```
[
  {
    "_id": "69cadf9eda6b3bf6af3bfe29",
    "ProductID": 1001,
    "Name": "NVIDIA RTX 4090",
    "UnitPrice": 1599.99,
    "StockQuantity": 12,
    "Description": "High-end GPU with 24GB VRAM for 4K gaming."
  },
  {
    "_id": "69cadf9eda6b3bf6af3bfe2f",
    "ProductID": 1007,
    "Name": "NZXT H7 Flow Case",
    "UnitPrice": 129.99,
    "StockQuantity": 22,
    "Description": "Mid-tower ATX case with high-airflow mesh front."
  },
  {
    "_id": "69cadf9eda6b3bf6af3bfe30",
    "ProductID": 1008,
    "Name": "Noctua NH-D15",
    "UnitPrice": 109.95,
    "StockQuantity": 45,
    "Description": "Dual-tower flagship CPU air cooler."
  },
  {
    "_id": "69cadf9eda6b3bf6af3bfe62",
    "ProductID": 1058,
```


Download

Response headers

```
content-length: 1867
content-type: application/json
date: Thu,02 Apr 2026 17:58:25 GMT
server: uvicorn
```

Paginate

GET /paginate Paginate

Parameters

Cancel

Name	Description
start_id * required integer (query)	1001
end_id * required integer (query)	1010

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/paginate?start_id=1001&end_id=1010' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/paginate?start_id=1001&end_id=1010
```

Server response

Code	Details
200	Response body


```
{
  "start_id": 1001,
  "end_id": 1010,
  "results": [
    {
      "_id": "69cadf9eda6b3bf6af3bfe29",
      "ProductID": 1001,
      "Name": "NVIDIA RTX 4090",
      "UnitPrice": 1599.99,
      "StockQuantity": 12,
      "Description": "High-end GPU with 24GB VRAM for 4K gaming."
    },
    {
      "_id": "69cadf9eda6b3bf6af3bfe2a",
      "ProductID": 1002,
      "Name": "AMD Ryzen 9 7950X",
      "UnitPrice": 549,
      "StockQuantity": 25,
      "Description": "16-core 32-thread enthusiast desktop processor."
    },
    {
      "_id": "69cadf9eda6b3bf6af3bfe2b",
      "ProductID": 1003,
      "Name": "Samsung 990 Pro 2TB",
      "UnitPrice": 179.99,
      "StockQuantity": 40,
      "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads."
```


Download

Response headers

```
content-length: 1821
content-type: application/json
date: Thu,02 Apr 2026 17:59:33 GMT
server: uvicorn
```

Responses

Convert

GET /convert/{product_id} Convert

Parameters

Cancel

Name	Description
product_id * required integer (path)	1001

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/convert/1001' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/convert/1001
```

Server response

Code	Details
200	Response body

```
{
  "Name": "NVIDIA RTX 4090",
  "UnitPrice_USD": 1599.99,
  "UnitPrice_EUR": 1379.19,
  "ExchangeRate": 0.862
}
```

Download

Response headers

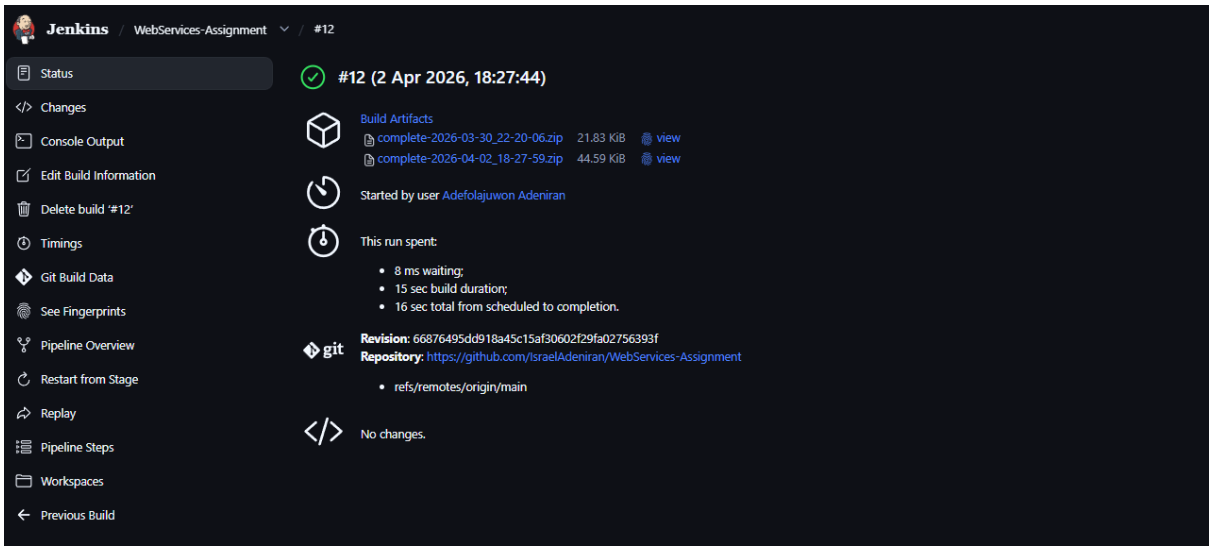
```
content-length: 95
content-type: application/json
date: Thu,02 Apr 2026 18:12:16 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value | Schema	No links
422	Validation Error	No links

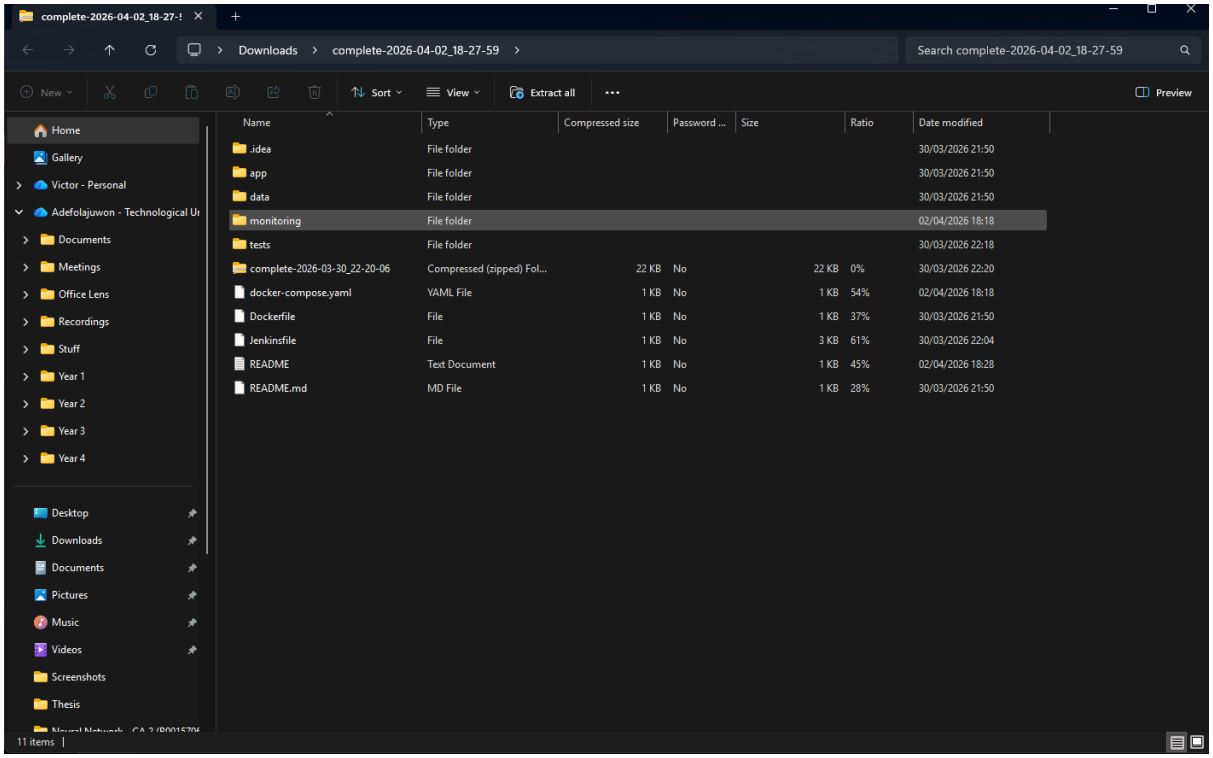
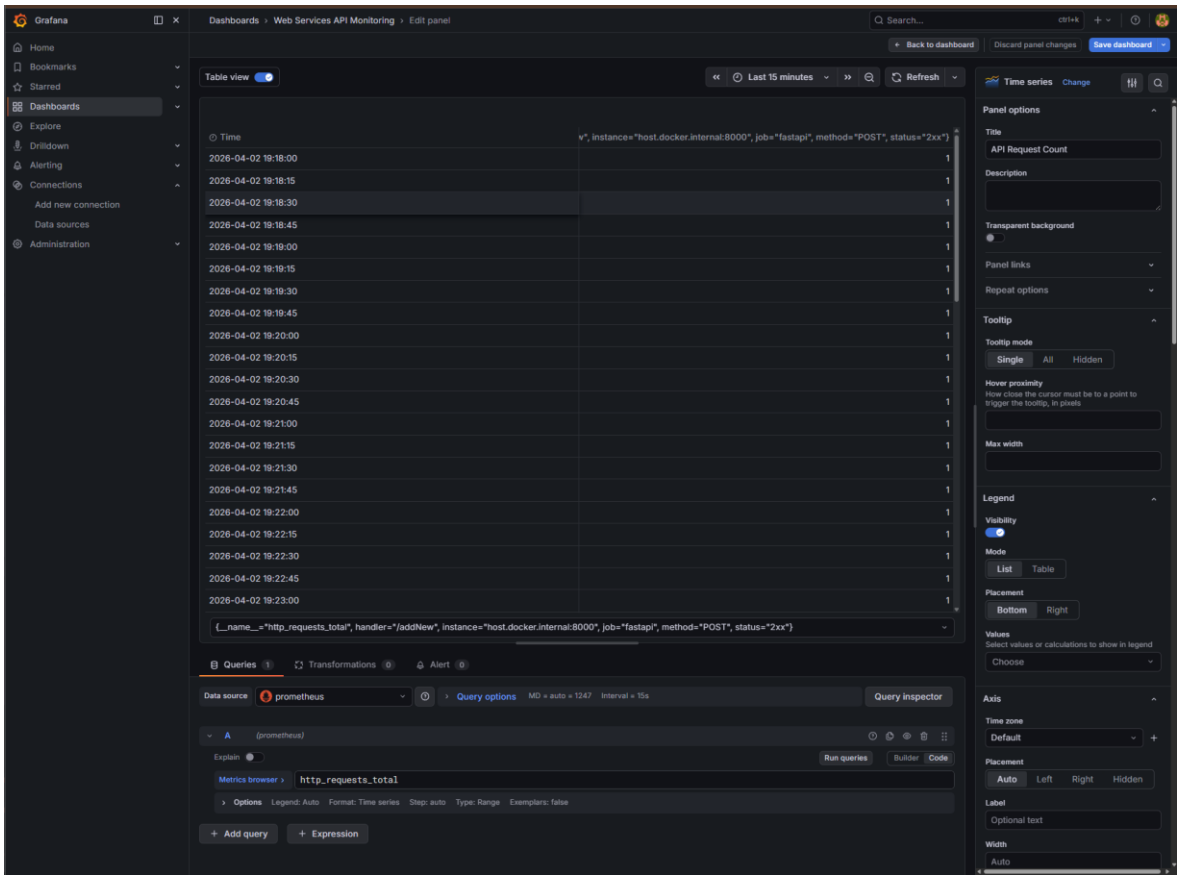
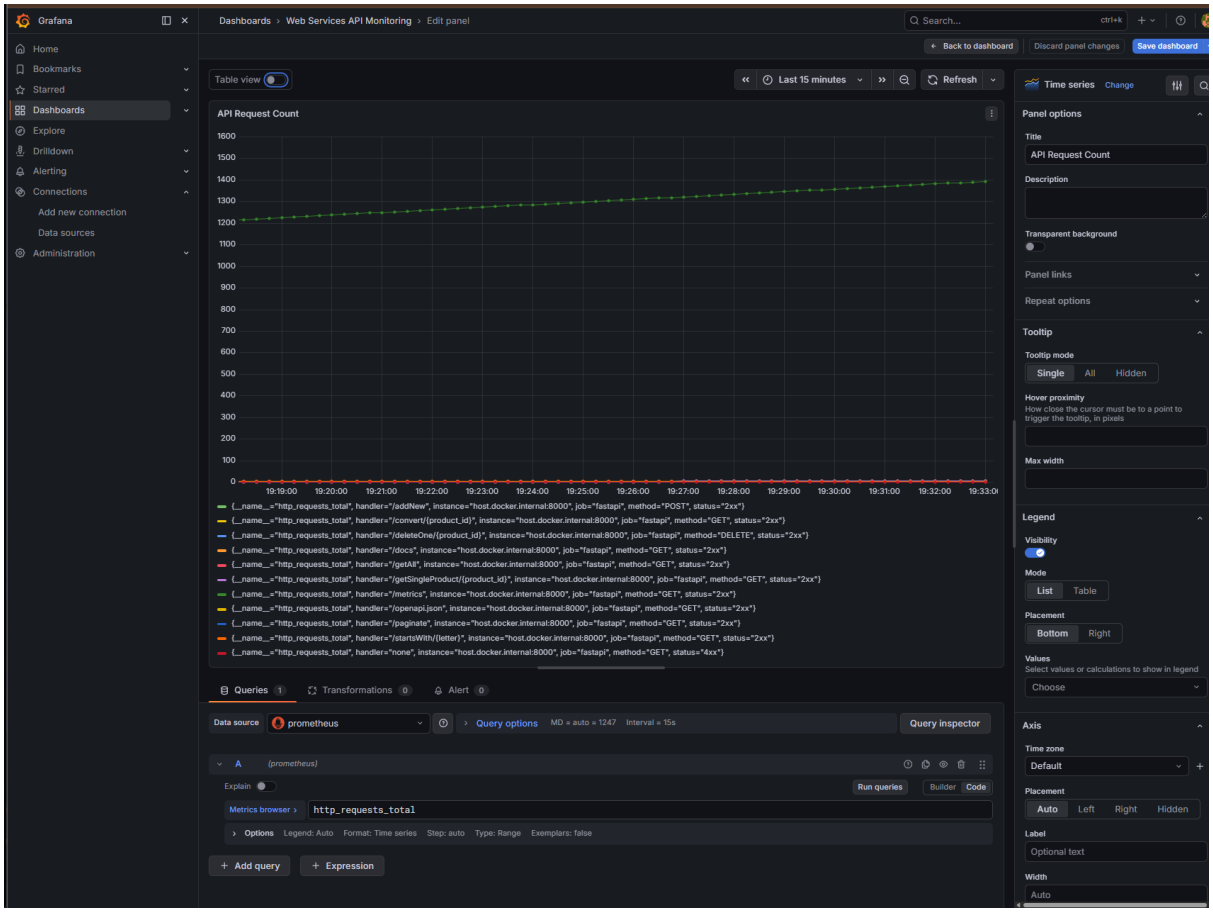
```
"string"
```

5. Jenkins Pipeline Screenshots




```
newman

Web Services Assignment Tests

→ Get Single Product
  GET http://host.docker.internal:8000/getSingleProduct/1001 [200 OK, 300B, 29ms]
  ✓  Status code is 200

→ Get All Products
  GET http://host.docker.internal:8000/getAll [200 OK, 34.88kB, 35ms]
  ✓  Status code is 200

→ Add New Product
  POST http://host.docker.internal:8000/addNew [200 OK, 165B, 26ms]
  ✓  Status code is 200

→ Delete Product
  DELETE http://host.docker.internal:8000/deleteOne/9999 [200 OK, 167B, 15ms]
  ✓  Status code is 200

→ Starts With
  GET http://host.docker.internal:8000/startsWith/N [200 OK, 1.99kB, 12ms]
  ✓  Status code is 200

→ Paginate
  GET http://host.docker.internal:8000/paginate?start_id=1001&end_id=1010 [200 OK, 1.95kB, 12ms]
  ✓  Status code is 200

→ Convert Price
  GET http://host.docker.internal:8000/convert/1001 [200 OK, 220B, 663ms]
  ✓  Status code is 200

┌─────────────────────────┬──────────────────┬──────────────────┐
│                         │         executed │           failed │
├─────────────────────────┼──────────────────┼──────────────────┤
│              iterations │                1 │                0 │
├─────────────────────────┼──────────────────┼──────────────────┤
│                requests │                7 │                0 │
├─────────────────────────┼──────────────────┼──────────────────┤
│            test-scripts │                7 │                0 │
├─────────────────────────┼──────────────────┼──────────────────┤
│      prerequest-scripts │                0 │                0 │
├─────────────────────────┼──────────────────┼──────────────────┤
│              assertions │                7 │                0 │
├─────────────────────────┴──────────────────┴──────────────────┤
│ total run duration: 899ms                                      │
├────────────────────────────────────────────────────────────────┤
│ total data received: 38.79kB (approx)                          │
├────────────────────────────────────────────────────────────────┤
│ average response time: 113ms [min: 12ms, max: 663ms, s.d.: 224ms] │
└────────────────────────────────────────────────────────────────┘
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Stop)
[Pipeline] sh
+ docker stop webservices-container
```

6. Zip File created by Jenkins



7. A-Grade Component